

Sortie (솔티)

OCR 및 온디바이스 멀티모달 분류 모델을 활용한
맞춤형 스크린샷 아카이빙 시스템



202255506 공다은

202255573 오예린

202255614 정채운

지도교수 탁성우

2026년 9월

Contents

1	서론	4
1.1	연구 배경: 패러다임 전환과 다크 데이터	4
1.2	상용 솔루션의 한계	4
1.3	연구 목표 및 차별점	4
2	연구 배경	6
2.1	스크린샷 기반 비정형 데이터	6
2.2	OCR 기술 선택	6
2.3	온디바이스 텍스트 분류	6
2.4	TF-IDF와 Character N-gram	7
2.5	NER와 개인정보 보호	7
2.6	암호화 방식	8
2.7	LLM과 Structured Output	8
2.8	SQLite와 JSON	9
2.9	선행연구 및 기술 선택의 시사점	9
3	연구 내용	10
3.1	전체 시스템 아키텍처	10
3.2	모바일 클라이언트	11
3.2.1	이미지 수집과 OCR	11
3.2.2	온디바이스 대분류	11
3.2.3	기프트콘 사전 처리	11
3.3	개인정보 보호 파이프라인	12
3.3.1	1단계: NER 기반 탐지	12
3.3.2	2단계: Regex 기반 탐지	12
3.3.3	암호화 및 서버 전달	12
3.4	서버 시스템	13
3.4.1	FastAPI 분석 API	13
3.4.2	LLM 호출과 재시도	14

3.4.3	동시성 제어	14
3.5	카테고리별 정보 구조화	14
3.5.1	SCHEDULE	14
3.5.2	PLACE	15
3.5.3	WISHLIST	16
3.5.4	MEMO	17
3.6	Validator와 데이터 품질 관리	18
3.7	로컬 DB와 저장 구조	18
3.8	검색 및 필터링	19
3.9	이미지 저장 및 생명주기	19
3.10	도메인별 Vision-to-Action 연동	20
3.10.1	WISHLIST: 네이버 쇼핑 최저가 검색 연동	20
3.10.2	PLACE: 카카오맵 다이렉트 길찾기 연동	20
3.10.3	SCHEDULE: 카카오 톡캘린더 자동 등록 및 알림	20
3.11	다중 이미지 처리	21
3.12	사용자 검토 및 상태 관리	21
4	연구 결과 분석 및 평가	23
4.1	평가 목적	23
4.2	평가 데이터 및 방법	23
4.3	평가 기준	23
4.4	카테고리별 처리 시간	24
4.5	SCHEDULE 추출 성능	24
4.6	PLACE 추출 성능	25
4.7	WISHLIST 추출 성능	25
4.8	MEMO 추출 성능	26
4.9	종합 성능 비교	26
4.10	LLM 토큰 사용량 및 비용	27
4.11	다중 처리 및 서버 동시성	27
4.12	개인정보 보호 평가	27
4.13	개발 과정의 문제와 개선	28

4.13.1	기프트콘 OCR 텍스트 축약 문제	28
4.13.2	지역명 정규화 문제	28
4.13.3	LLM 출력의 불확실성	28
5	결론 및 향후 연구 방향	29
5.1	연구 결과 요약	29
5.2	정량 평가의 의의	29
5.3	연구의 한계	29
5.4	연구 결과의 학술적·산업적 의의	30
5.5	비즈니스 모델 및 확장 가능성	30
5.5.1	B2C 제휴 커머스(Affiliate) 및 타겟 마케팅	30
5.5.2	B2B 엔터프라이즈 SDK 라이선싱	30
5.5.3	OS 통합형 지능형 비서 (Agentic Gateway)	31
5.5.4	보안 및 AI 기술의 고도화	31
6	참고 문헌	32

1. 서론

1.1. 연구 배경: 패러다임 전환과 다크 데이터

현대인에게 스마트폰 스크린샷은 가장 직관적인 정보 수집 도구이다. 사용자는 웹페이지, 메신저, SNS 등에서 발견한 가치 있는 정보를 별도의 메모 앱을 켜지 않고 즉각적으로 캡처한다. 그러나 이러한 방식은 기록의 편의성에도 불구하고, 텍스트가 아닌 '이미지'의 형태로 저장되므로 검색이나 편집이 불가능한 '데이터의 무덤(Dark Data)'을 양산하는 문제를 안고 있다. 기존 스마트폰 환경에서 스크린샷은 수동적으로 보관되는 픽셀 덩어리(Passive Image Hoarding)에 불과했다.

시간이 지나면 사용자는 수많은 이미지 속에서 필요한 정보를 다시 찾아야 하며, 정보를 발견하더라도 이를 다른 애플리케이션(지도, 캘린더, 쇼핑물 등)에 직접 입력해야 하는 인지적·물리적 마찰(Cognitive Load & Friction)이 발생한다. 본 연구는 이러한 문제를 해결하기 위해 스크린샷을 단순한 이미지 보관이 아닌 '실행 가능한 지식 베이스(Actionable Knowledge Base)'로 전환하는 시스템을 제안한다.

1.2. 상용 솔루션의 한계

최근 이러한 문제를 해결하기 위해 다양한 서비스가 등장했으나, 각각 뚜렷한 한계를 지닌다. 첫째, **단순 갤러리 정리 앱(예: Slidebox)**은 스크린샷을 폴더별로 분류할 뿐, 이미지 내부의 의미를 추출하거나 다른 서비스와 연동하지 못하는 '수동적 도구'에 머물러 있다. 둘째, **클라우드 기반 AI 보관함(예: mymind)**은 강력한 분류 성능을 제공하지만, 스크린샷 원본을 무조건 서버로 전송해야 하므로 민감한 개인정보(신분증, 계좌번호 등)의 유출 위험이 따르는 프라이버시 취약점을 지닌다. 셋째, **완전 온디바이스 AI(예: Google Pixel Screenshots)**는 로컬 환경에서 프라이버시를 보호하며 정보를 처리하지만, 모바일 기기의 연산 한계로 인해 자연어 문맥의 깊은 이해나 복잡한 정보 추출에서는 한계를 보이며 종종 클라우드 처리로 회귀(Fallback)하는 불안정한 모습을 보인다.

따라서 본 연구는 '온디바이스의 안전성'과 '클라우드의 높은 지능'을 결합한 하이브리드 아키텍처를 채택하였다. 특히, 개인정보 보호를 시스템 설계의 기본 요소로 삼는 '선제적 프라이버시(Privacy-by-Design)' 원칙을 적용하여 차별화된 솔루션을 구축하였다.

1.3. 연구 목표 및 차별점

본 프로젝트(솔티, Sortie)의 목표는 사용자가 스크린샷을 찍는 즉시, 그 안에 담긴 파편화된 정보를 카테고리(일정, 장소, 위시리스트, 메모)별로 구조화(Datafication)하고 사용자 입력 마찰을 최소화(Zero-Friction)하는 것이다. 솔티의 핵심 차별점은 다음과 같다.

- 1) **이중 방어망 기반 하이브리드 처리:** 로컬 기기(온디바이스)에서 초경량 모델을 통해 대분류

를 수행하고 개인정보를 즉각 암호화한 뒤, 안전하게 마스킹된 데이터만 클라우드 LLM으로 전송해 심층 구조화를 수행한다.

- 2) **Zero-Friction 액션 파이프라인**: 추출된 정보를 단순 저장하는 것에 그치지 않고, 네이버 쇼핑 최저가 검색, 카카오맵 길찾기, 카카오 톡캘린더 자동 등록 등 각 도메인에 맞는 외부 서비스로 즉시 연결하는 액션 파이프라인을 구축했다.

본 연구는 이러한 파이프라인을 실제 애플리케이션으로 구현하고, 평가 데이터에 근거하여 시스템의 효용성과 성능을 검증하는 데 그 의의가 있다.

2. 연구 배경

2.1. 스크린샷 기반 비정형 데이터

스크린샷은 생성 과정에서 별도의 데이터 스키마를 갖지 않는다. 같은 이미지 안에 날짜, 가격, 장소, 설명, 버튼, 메뉴, 광고, 상태바 등이 동시에 존재할 수 있다. 특히 모바일 화면은 본문과 UI 요소가 섞여 있어 OCR 결과에도 불필요한 문자열이 포함될 가능성이 있다.

따라서 스크린샷 처리에서는 다음 세 단계가 필요하다.

1. 이미지에서 텍스트를 추출하는 OCR 단계
2. 텍스트의 의미적 종류를 판단하는 분류 단계
3. 카테고리별 필드로 변환하는 구조화 단계

솔티는 이 세 단계에 개인정보 보호 단계를 추가하여 “OCR → 분류 → 민감정보 보호 → 구조화”의 파이프라인을 구성한다.

2.2. OCR 기술 선택

프로젝트의 Flutter 클라이언트는 `google_mlkit_text_recognition` 패키지를 사용한다. OCR을 모바일 클라이언트에서 수행함으로써 이미지 전체를 서버에 전송하지 않고 텍스트 추출을 먼저 수행할 수 있다. 이 구조는 네트워크 전송량을 줄이는 동시에 개인정보 보호 계층을 서버 요청보다 앞에 배치할 수 있다는 장점이 있다.

OCR 결과는 단순 문자열이 아니라 이후 분류 및 마스킹의 입력으로 사용된다. 실제 코드에서는 상단 상태바의 시간, 날짜, 통신사 등의 문자열이 본문에 혼입되는 것을 줄이기 위해 OCR 텍스트의 초기 두 줄을 대상으로 상태바 패턴을 검사하여 제거하는 로직도 포함한다.

2.3. 온디바이스 텍스트 분류

프로젝트의 `OnDeviceTextClassifier`는 TFLite Interpreter를 이용하여 `classifier_demo.tflite`를 로드한다. 함께 제공되는 `vocab.json`, `idf.json`, `classifier_label_map.json`을 이용하여 텍스트를 모델 입력으로 변환한다.

전처리는 다음과 같이 구현되어 있다.

1. 한글, 영문, 숫자를 제외한 특수문자를 공백으로 치환
2. 연속된 공백을 하나로 정규화

3. 소문자 변환
4. 공백을 기준으로 단어 분리
5. 각 단어에 공백을 붙인 뒤 character bigram과 trigram 생성
6. 프로젝트 vocabulary에 존재하는 n-gram만 TF-IDF 벡터에 반영
7. L2 normalization 수행
8. TFLite 모델에 입력하여 네 개 클래스의 확률 계산
9. 최댓값의 클래스를 최종 카테고리로 선택

이 방식은 형태소 분석기와 같은 별도의 무거운 언어 처리기를 사용하지 않고, 모델 학습 시 사용한 특징 추출 방식을 모바일 코드에 이식한다는 특징이 있다. Character N-gram은 단어 전체가 정확히 일치하지 않아도 부분적인 문자열 특징을 사용할 수 있으므로 OCR 오류나 형태 변화에 대한 보완 수단으로 활용될 수 있다.

2.4. TF-IDF와 Character N-gram

TF-IDF는 문서 내에서 자주 나타나지만 전체 문서 집합에서는 상대적으로 희소한 단어 또는 특징에 더 높은 가중치를 부여하는 방식이다. 본 프로젝트에서는 일반적인 단어 단위 TF-IDF가 아니라 단어 경계를 고려한 character bigram과 trigram을 사용한다.

문서 d 에서 특징 t 의 TF-IDF는 개념적으로 다음과 같이 표현할 수 있다.

$$\text{TFIDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

실제 모바일 코드에서는 프로젝트의 `idf.json`에 저장된 IDF 값을 불러와 각 n-gram의 등장 횟수와 곱한다. 이후 전체 벡터를 L2 정규화한다.

이러한 설계는 모델과 모바일 전처리의 불일치를 줄인다는 점에서 중요하다. 서버에서 사용하는 별도의 자연어 처리 방식이 아니라 학습 시 사용한 특징을 동일한 형태로 생성하기 때문에 TFLite 모델이 기대하는 입력 공간을 유지할 수 있다.

2.5. NER와 개인정보 보호

정규표현식은 일정한 패턴을 가진 개인정보 탐지에 효과적이지만, 사람 이름과 같이 문맥에 의존하는 정보에는 한계가 있다. 반대로 NER 모델은 문맥을 고려할 수 있지만 모든 번호 형식에 대해 동일한 강점을 가지지는 않는다.

솔티의 MaskingHelper는 이 두 방식을 결합한다. 먼저 온디바이스 NER 모델이 WordPiece 기반 토큰라이저를 통해 최대 128개 토큰 길이의 입력을 구성하고 TFLite 추론을 수행한다. 현재 코드에서 마스킹 대상으로 사용하는 NER 엔티티는 PER와 PHONE이다.

이후 Regex 계층에서는 주민등록번호, 카드번호, CVC/CVV, 쿠폰 번호, 계좌번호, 전화번호, 예약번호, 여권번호 및 MRZ, 운전면허번호, 신분증 주소와 날짜 등 여러 패턴을 별도로 처리한다. 즉, “AI가 전부 탐지한다”는 구조가 아니라 NER와 패턴 기반 탐지를 역할별로 결합한 구조이다.

2.6. 암호화 방식

민감정보의 치환에는 CryptoHelper가 사용된다. 코드에서는 32바이트의 랜덤 키를 생성하여 FlutterSecureStorage에 저장하고, AES-256-CBC 방식으로 문자열을 암호화한다. 각 암호화마다 16바이트의 랜덤 IV를 생성하고 IV와 암호문을 결합한 뒤 Base64로 인코딩한다. 결과는 다음과 같은 형태의 문자열로 반환된다.

`[ENC:Base64_IV_Ciphertext]`

서버에 전달되는 OCR 텍스트에는 이러한 토큰이 포함될 수 있으며, 클라이언트의 복호화 함수는 `[ENC:...]` 패턴을 찾아 로컬에서 다시 평문으로 복원한다.

본 시스템은 서버 전송 전 기밀성(Confidentiality) 보장을 위해 AES-CBC를 적용하였으며, 향후 데이터 무결성 검증을 위해 AES-GCM과 같은 인증 암호화(Authenticated Encryption) 방식의 도입을 고려할 수 있다.

2.7. LLM과 Structured Output

스크린샷 정보 중 날짜나 가격처럼 정형적인 정보는 정규표현식으로 처리하기 쉽지만, “이 장소가 어느 식당인지”, “이 화면에서 상품명이 무엇인지”, “이 메모의 핵심 제목이 무엇인지”와 같은 정보는 문맥 이해가 필요하다.

따라서 서버에서는 카테고리별 프롬프트를 사용하여 LLM에게 필요한 필드를 추출하도록 한다. SCHEDULE, PLACE, WISHLIST, MEMO마다 서로 다른 필드 정의와 예외 규칙을 사용한다.

예를 들어 SCHEDULE 프롬프트에서는 체크인과 체크아웃을 하나의 일정으로 연결하고, 기프트콘의 유효기간을 `expires.at`에 저장하며, 취소 마감일을 일반 일정의 시작일이나 만료일로 오인하지 않도록 명시한다. PLACE에서는 여러 장소가 나열될 경우 각각을 객체로 분리하고, WISHLIST에서는 여러 상품을 별도의 객체로 분리하도록 지시한다.

2.8. SQLite와 JSON

모바일 클라이언트와 서버 모두 SQLite를 사용한다. 클라이언트의 `screenshots` 테이블은 `id`, `asset_id`, `type`, `confidence`, `fields`, `image_path`, `status`, `created_at`, `updated_at` 등의 정보를 저장한다.

세부 추출 필드는 카테고리에 따라 달라지므로 고정 컬럼 대신 `fields`에 JSON 문자열을 저장한다. 서버 DB에는 이미지 해시를 저장하여 동일 이미지의 분석 결과를 조회할 수 있도록 하며, 알림 기능을 위해 기기 토큰과 알림 내역 테이블도 존재한다.

2.9. 선행연구 및 기술 선택의 시사점

기존 초안에서 참고한 선행연구는 개인정보 보호형 LLM 처리, 온디바이스 모델의 한계, 프롬프트 엔지니어링 등의 관점에서 본 프로젝트의 설계 방향을 설명하는 데 활용할 수 있다. 다만 최종보고서에서는 실제로 확인되지 않은 성능이나 특정 연구의 결론을 프로젝트 결과처럼 서술하지 않고, “기술 선택에 참고한 배경”으로 한정한다.

기술	프로젝트에서의 역할	선택 이유
ML Kit OCR	이미지 → 텍스트	모바일에서 직접 수행하여 서버 전송 전 전처리 가능
TFLite 분류기	네 가지 대분류	모바일에서 빠른 추론 및 네트워크 비의존적 분류 가능
Character N-gram + TF-IDF	분류 입력 특징	프로젝트의 학습 특징을 모바일에서 재현
NER + Regex	민감정보 탐지	문맥 기반 탐지와 명시적 패턴 탐지를 결합
AES-256-CBC	민감 문자열 보호	서버 전송 전 원문 값을 암호문 토큰으로 치환
LLM SQLite + JSON	상세 필드 추출 결과 저장	비정형 정보의 문맥 기반 구조화 카테고리별 상이한 필드를 유연하게 저장
FastAPI	분석 API	모바일 클라이언트와 LLM 처리 계층 분리

Table 1: 주요 기술의 선택 및 역할

3. 연구 내용

3.1. 전체 시스템 아키텍처

솔티의 전체 처리 흐름은 모바일 클라이언트와 서버의 역할을 분리한 구조로 구성된다.

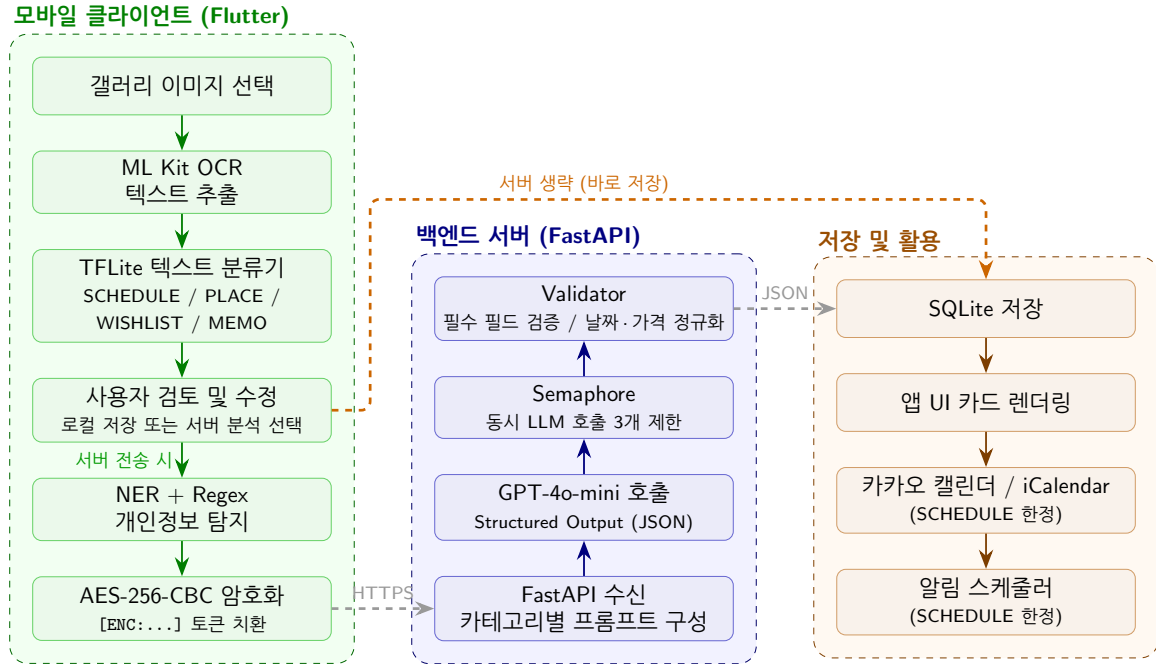


Figure 1: 솔티(Sortie) 전체 시스템 아키텍처

사용자가 이미지를 선택하면 Flutter 클라이언트가 이미지에서 OCR 텍스트를 획득하고, 이후 TFLite 로컬 분류기가 텍스트를 SCHEDULE, PLACE, WISHLIST, MEMO 중 하나의 대분류로 예측한 1차 초안을 제시한다. 이때 사용자는 분류된 카테고리나 OCR 텍스트를 수동으로 검토 및 수정할 수 있으며, 이 결과를 바탕으로 로컬 보관함에 바로 저장할 것인지, 혹은 심층 구조화 데이터를 얻기 위해 서버로 전송하여 LLM 분석을 받을 것인지 선택할 수 있다.

사용자가 서버 전송을 선택한 경우에 한해, 텍스트는 로컬 NER 및 Regex 계층을 통과하며 민감한 구간이 암호화 토큰으로 변환된다. 이후 클라이언트는 분류 결과와 마스킹된 OCR 텍스트를 FastAPI 서버의 분석 API에 전달하고, 서버는 카테고리별 프롬프트와 입력 텍스트를 LLM에 전달하여 반환된 JSON을 검증 및 보강한 뒤 클라이언트에 최종 반환한다. 그림 1은 이러한 선택적 전송 흐름을 포함한 솔티의 전체 시스템 아키텍처를 보여준다.

3.2. 모바일 클라이언트

3.2.1. 이미지 수집과 OCR

프로젝트는 `image_picker`, `photo_manager`, `wechat_assets_picker`, `image_cropper` 등의 패키지를 사용한다. 이를 통해 갤러리 이미지 선택과 이미지 관련 작업을 수행한다.

OCR은 `google_mlkit_text_recognition`을 이용한다. OCR 결과는 화면에서 초안 데이터로 관리되며 이후 온디바이스 분류 및 AI 분석 요청의 입력으로 사용된다.

상단 상태바 제거 로직은 OCR 결과에 포함될 수 있는 현재 시간, 날짜, 통신사 등의 정보를 본문 정보와 구분하기 위한 것이다. 이 단계는 정보 추출 정확도를 높이기 위한 단순한 노이즈 제거 단계로 볼 수 있다.

3.2.2. 온디바이스 대분류

온디바이스 분류기는 다음 네 가지 클래스를 반환한다.

Index	Type	의미
0	SCHEDULE	일정, 예약, 기한 등 시간 중심 정보
1	PLACE	장소, 맛집, 위치 등 장소 중심 정보
2	WISHLIST	상품, 쇼핑, 구매 관심 정보
3	MEMO	기타 기록 및 참고 정보

Table 2: 온디바이스 분류기의 대분류

모델이 초기화되지 않았거나 전처리 결과가 비어 있는 경우 코드의 폴백 값은 MEMO이다. 또한 주민등록증, 운전면허증, 여권 등의 신분증 텍스트가 감지되는 경우에는 민감한 문서의 내용을 일반적인 일정이나 장소로 잘못 분류하지 않도록 MEMO 카테고리 강제로 강제하는 로직이 존재한다.

3.2.3. 기프티콘 사전 처리

프로젝트에는 기프티콘 관련 OCR을 처리하기 위한 별도의 휴리스틱이 존재한다. 기프티콘, 쿠폰, 바코드, 교환권, 모바일상품권, 선물하기, 교환처 등의 키워드를 이용하여 해당 문서의 특성을 판단한다.

코드에는 유효기간, 교환처, 상품명을 탐색하는 `normalizeGifticon` 함수가 존재한다. 초기에는 기프티콘 OCR 텍스트를 세 줄로 축약하는 방식을 고려하였으나, 정보 손실을 방지하기 위해 현재는 원본 OCR 텍스트 전체를 보존하여 LLM 입력에 활용하도록 개선하였다.

3.3. 개인정보 보호 파이프라인

3.3.1. 1단계: NER 기반 탐지

NerClassifier는 앱 시작 시 `ner_model.tflite`, `vocab.txt`, `ner_label_map.json`을 로드한다. 토큰나이저는 WordPiece 방식으로 단어를 분해하고 최대 128개 토큰 길이에 맞춘다.

모델은 각 토큰에 대해 라벨을 출력하며, 현재 코드에서 B-PER, I-PER, B-PHONE, I-PHONE 등의 라벨 중 PER과 PHONE 엔티티가 마스킹 대상이다. 탐지된 토큰의 원문 offset을 이용하여 해당 문자열 범위를 찾고, 겹치거나 인접한 범위를 병합한 후 역순으로 암호화한다.

3.3.2. 2단계: Regex 기반 탐지

NER 이후에는 여러 정형 패턴을 Regex로 검사한다.

대상	구현 특징
주민등록번호	6자리 생년월일 + 1-8로 시작하는 7자리 패턴
카드번호	공백/하이픈을 고려한 14-16자리 계열 패턴 및 카드 키워드 검증
CVC/CVV	CVC, CVV, 보안코드 등의 키워드와 3-4자리 숫자 결합
쿠폰번호	12-16자리 숫자와 기프티콘 관련 키워드의 결합
계좌번호	여러 구간의 숫자와 하이픈 패턴, 은행 관련 키 워드 검증
전화번호	01X 및 지역번호 기반 전화번호 패턴
예약번호	예매번호, 예약번호, 티켓번호 등의 키워드와 영 숫자 조합
여권번호	여권 키워드 및 영숫자 번호 패턴
여권 MRZ	“i” 문자를 포함하는 판독 영역 후보를 인접 줄과 함께 탐지
운전면허번호	지역 코드 및 숫자 조합 패턴
신분증 주소/날짜	신분증 문서가 확인된 경우 주소 및 날짜 패턴을 추가 처리

3.3.3. 암호화 및 서버 전달

탐지된 값은 `CryptoHelper().encryptSensitive()`를 통해 암호화된다. 따라서 LLM 요청의 텍스트에는 원래의 번호 대신 `[ENC:...]` 형태의 토큰이 들어간다.

이 구조의 중요한 점은 “LLM이 개인정보를 삭제한다”가 아니라, 로컬에서 민감 문자열을 변환한다는 것이다.

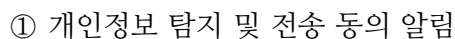


Figure 2: 온디바이스 개인정보 탐지 및 암호화 서버 전송 과정

3.4. 서버 시스템

3.4.1. FastAPI 분석 API

백엔드에는 FastAPI 애플리케이션이 구성되어 있으며 분석 관련 라우터와 알림, 결과 조회 등의 기능이 분리되어 있다. 모바일 클라이언트는 `/api/analyze/v2` 엔드포인트를 이용하여 OCR 텍스트와 분류 결과를 전달한다.

단건 요청은 JSON 형태로 다음과 같은 개념적 구조를 갖는다.

```
{
  "ocr_text": "... masked OCR text ...",
  "type": "SCHEDULE",
  "masked_tokens": []
}
```

다중 요청에서는 items 배열을 사용하여 여러 분석 항목을 한 번에 전달한다.

3.4.2. LLM 호출과 재시도

서버의 LLM 클라이언트는 모델 호출을 별도 모듈로 분리한다. LLM 호출은 최대 재시도 횟수를 고려하는 구조를 갖고 있으며, 호출이 최종적으로 실패하면 MEMO, confidence 0, 빈 fields 및 수동 입력 필요 메시지를 포함한 폴백 결과를 반환한다.

본 시스템은 LLM 호출 계층을 별도 모듈로 분리하여 모델 버전이나 API 공급자가 변경되더라도 유연하게 대응할 수 있도록 설계하였다.

3.4.3. 동시성 제어

서버에는 `MAX_CONCURRENT_LLM = 3`으로 정의된 Semaphore가 있다. 여러 요청이 동시에 들어오더라도 동시에 실행되는 LLM 호출 수를 제한하여 API Rate Limit이나 서버 측 자원 과부하 가능성을 줄이는 역할을 한다.

여기서 $C_{\max} = 3$ 은 동시에 LLM API를 호출할 수 있는 최대 호출 수를 의미한다. 실제 처리 시간은 네트워크 상태와 LLM 응답 시간에 따라 달라지므로 이 값 자체를 “20건 처리 시간이 항상 일정하다”는 의미로 해석해서는 안 된다.

3.5. 카테고리별 정보 구조화

3.5.1. SCHEDULE

SCHEDULE은 시간에 따라 행동이 필요한 정보를 표현한다. 서버 프롬프트와 validator는 다음과 같은 필드를 다룬다.

- title
- start_at, end_at
- expires_at
- start_time, end_time
- exchange_place
- participants
- recurrence
- cancellation_deadline
- sub_type

세부 유형은 GIFTICON, SUBSCRIPTION, APPOINTMENT, TICKET, DEADLINE, DELIVERY 등을 사용한다. validator는 유형에 따라 keep_photo, reminder_days, calendar_type 등의 값을 보장한다.

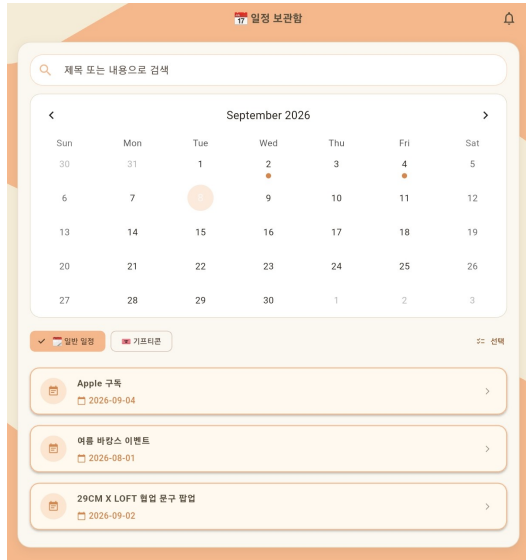


Figure 3: SCHEDULE(일정) 보관함 UI

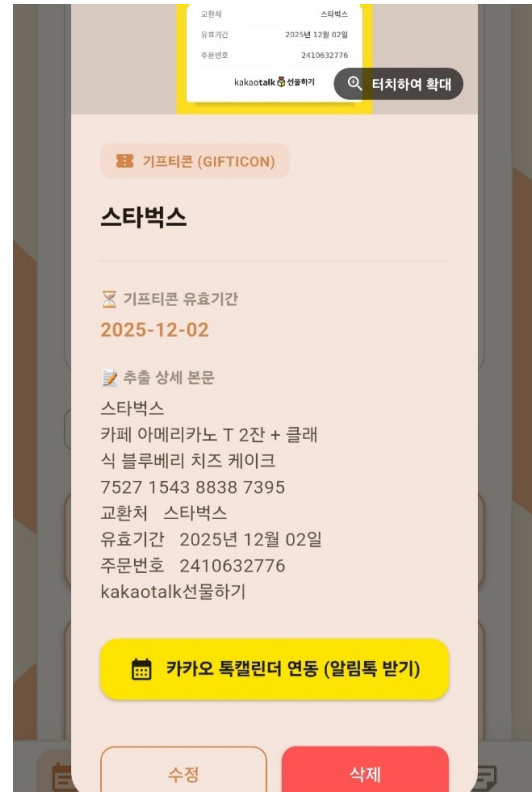


Figure 4: 카카오톡 캘린더 연동 화면

3.5.2. PLACE

PLACE는 장소의 이름과 지역 정보를 중심으로 구조화된다. 서버의 후처리에서는 지역명을 15개 표준 광역 단위로 정규화하고, “해운대”가 포함된 경우 부산으로 처리하는 예외도 존재한다.

또한 map_ready는 주소 또는 지역 정보가 존재하는지를 이용하여 지도 관련 UI를 사용할 수 있는지 판단한다.

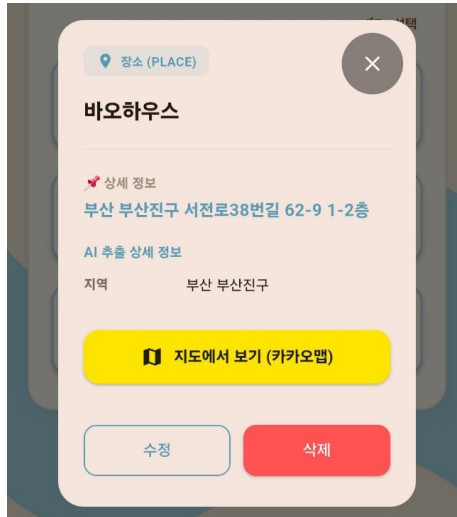


Figure 5: 카카오맵 연동 기능

3.5.3. WISHLIST

WISHLIST는 상품 단위의 정보로 구조화된다. 주요 필드는 상품명, 가격, 옵션, 판매처, URL이다. 여러 상품이 하나의 OCR 텍스트에 포함될 경우 서버 프롬프트는 각각의 객체로 분리하도록 설계되어 있다.

validator의 후처리에서는 가격 문자열에서 숫자와 소수점을 제외한 문자를 제거하여 수치형으로 정규화한다. keep_photo의 기본값은 true이다.

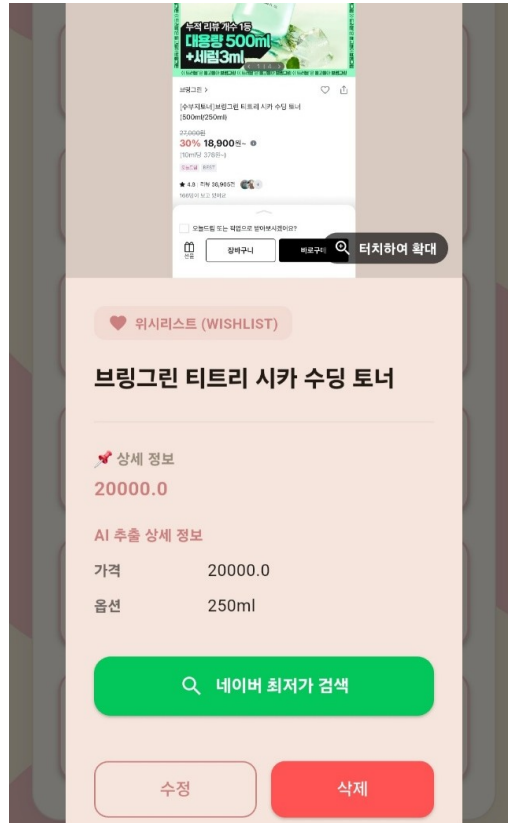


Figure 6: 최저가 검색 제공

3.5.4. MEMO

MEMO는 일정, 장소, 상품에 해당하지 않는 일반적인 참고 정보를 대상으로 한다. 서버 프롬프트에서는 RECIPE, NOVEL, CHECKLIST, ARTICLE, NOTE, OTHER 등의 `sub.type`을 사용한다.

예를 들어 RECIPE에서는 재료와 조리 순서, NOVEL에서는 작품명과 작가, CHECKLIST에서는 체크 항목 등의 필드를 추가로 다룬다. title이 존재하지 않는 경우 body 앞부분을 이용해 제목을 생성하는 후처리도 구현되어 있다. 또한, 보관함에 저장할 때 기본 제공되는 서브타입 외에 사용자가 직접 커스텀 `sub.type`을 정의하여 보다 유연하게 메모를 분류할 수도 있다.



Figure 7: MEMO(메모) 보관함 UI

3.6. Validator와 데이터 품질 관리

LLM은 확률적인 생성 모델이므로 JSON 형식이 맞더라도 필수 정보가 누락될 수 있다. 이를 보완하기 위해 validator가 별도로 존재한다.

검증 단계에서는 다음을 수행한다.

1. type이 허용된 네 가지 값인지 검사
2. confidence가 0과 1 사이인지 검사
3. 카테고리별 필수 필드 존재 여부 검사
4. 여러 객체가 반환되는 경우 각각 검사
5. SCHEDULE의 시작일/만료일 존재 여부 검사
6. PLACE의 name 또는 region 존재 여부 검사
7. 카테고리별 enrichment 수행
8. 누락된 필드를 `missing_fields`에 기록

따라서 서버의 역할은 단순히 LLM 결과를 그대로 반환하는 것이 아니라 “LLM 생성 → 구조 검증 → 후처리 및 보강 → API 응답”으로 확장된다.

3.7. 로컬 DB와 저장 구조

모바일 DB의 핵심 테이블은 `screenshots`이다.

필드	타입	역할
id	INTEGER	자동 증가 기본키
asset_id	TEXT	갤러리 원본 식별자
type	TEXT	SCHEDULE/PLACE/WISHLIST/MEMO
confidence	REAL	분석 신뢰도 저장
fields	TEXT	카테고리별 JSON 데이터
image_path	TEXT	보존된 이미지 경로
status	TEXT	DRAFT 등의 상태 관리
created_at	TEXT	생성 시각
updated_at	TEXT	수정 시각

Table 4: 모바일 screenshots 테이블 구조

서버 DB에는 별도의 `image_hash` 컬럼이 존재하며, 동일 이미지에 대한 분석 결과를 조회하는 함수가 구현되어 있다. 이를 통해 분석 결과 캐시 및 중복 분석 방지에 활용할 수 있는 기반을 제공한다.

3.8. 검색 및 필터링

서버의 결과 조회 기능은 `type`, `status`, `region`, `category` 및 자유 텍스트 `q`를 기준으로 결과를 검색할 수 있다. JSON으로 저장된 `fields` 내부 값에 대해 SQLite의 `json_extract`와 LIKE 검색을 사용하는 방식이 구현되어 있다.

이는 단순히 “DB에 결과를 저장한다”는 수준을 넘어 구조화 결과가 실제 애플리케이션의 검색 기준으로 사용될 수 있도록 한 것이다.

3.9. 이미지 저장 및 생명주기

앱 내부에는 SCHEDULE, PLACE, WISHLIST, MEMO별 디렉토리를 생성하는 `AppStorage`가 존재한다. 서버의 `validator`는 카테고리화 SCHEDULE의 세부 유형에 따라 `keep_photo` 기본값을 결정한다.

유형	keep_photo 기본값	의도
GIFTICON	true	바코드/쿠폰 이미지 보존
TICKET	true	예매 관련 원본 보존
WISHLIST	true	상품 이미지 보존
PLACE	false	텍스트 구조화 후 이미지 의존도 감소
MEMO	false	텍스트 중심 정보
APPOINTMENT	false	일정 구조화 중심
SUBSCRIPTION	false	갱신 정보 구조화 중심
DEADLINE	false	마감 정보 구조화 중심
DELIVERY	false	배송 일정 구조화 중심

Table 5: 코드에 정의된 keep-photo 기본 정책

3.10. 도메인별 Vision-to-Action 연동

본 프로젝트는 AI를 통해 추출된 비정형 데이터를 단순히 로컬 DB에 보관하는 것에 그치지 않고, 각 카테고리(도메인)의 특성에 맞는 외부 서비스와 즉각적으로 연계하여 사용자 입력 마찰(Zero-Friction)을 최소화하는 액션 파이프라인을 프론트엔드(Flutter) 단에 구축하였다.

3.10.1. WISHLIST: 네이버 쇼핑 최저가 검색 연동

위시리스트 보관함 카드 상세 화면에 외부 검색 연동 기능을 추가하여 즉각적인 가격 비교 및 구매 여정을 지원한다.

- **구현 방식:** AI가 추출한 `product_name`(상품명) 데이터를 URL 인코딩하여 네이버 쇼핑 검색 URL과 동적으로 결합한 후, `url_launcher` 패키지를 통해 외부 브라우저로 즉시 호출한다.
- **예외 처리:** 상품명에 공백이거나 '제목 없음'일 경우 실행을 막고 스낵바(SnackBar) 알림을 띄우는 방어 로직을 적용하였다.
- **기대 효과:** 캡처한 상품을 찾기 위해 브라우저를 켜고 product_name을 다시 타이핑해야 하는 번거로움을 없애 편의성을 극대화하였다.

3.10.2. PLACE: 카카오맵 다이렉트 길찾기 연동

장소 보관함 상세 화면에 지도 앱 연동 기능을 추가하여, 나만의 장소 저장소를 넘어 곧바로 네비게이션 및 길찾기의 선행 관문(Gateway)이 되도록 UX를 고도화하였다.

- **검색어 최적화:** 상호명(title)을 최우선 검색어로 사용되되, 상호명이 없을 경우 주소(extraInfo)로 대체하여 검색하도록 분기 처리하였다.
- **스마트 폴백(Fallback) 라우팅:** 사용자 기기에 카카오맵 앱이 설치되어 있는 경우 딥링크(`kakao://search?q=`)를 우선 호출하고, 앱이 없을 경우 웹 브라우저 지도로 우회하도록 try-catch 예외 처리를 구현하였다.

3.10.3. SCHEDULE: 카카오 톡캘린더 자동 등록 및 알림

일정 보관함 상세 화면에서 톡캘린더 연동을 지원하여 일정 잊음 방지 및 서비스 실용성을 향상시켰다. (기존 대기실에서 제공되던 기능을 보관함으로 확장 적용하였다.)

- **데이터 정규화 및 SDK 연동:** 추출된 텍스트 형태의 날짜 데이터에서 불필요한 특수문자를 제거하고 `DateTime` 객체로 파싱한다. 이후 카카오 SDK(`KakaoCalendarService`)의 `createEvent` API를 호출하여 추출된 제목과 일시 정보를 카카오톡 캘린더에 자동 등록한다.

- **서버 연계 및 스케줄러:** 서버 측에서는 별도로 iCalendar(.ics) 형식의 문자열을 생성하는 기능을 제공하며, 60초 간격으로 DB를 확인하여 오늘/내일이 기한인 SCHEDULE 및 GIFTICON 항목에 대한 알람 내역을 생성하는 백그라운드 스케줄러가 함께 동작한다.

3.11. 다중 이미지 처리

모바일 클라이언트는 여러 스크린샷을 선택하여 일괄 분석할 수 있으며, 코드에서는 요청을 20개 단위로 분할한다.

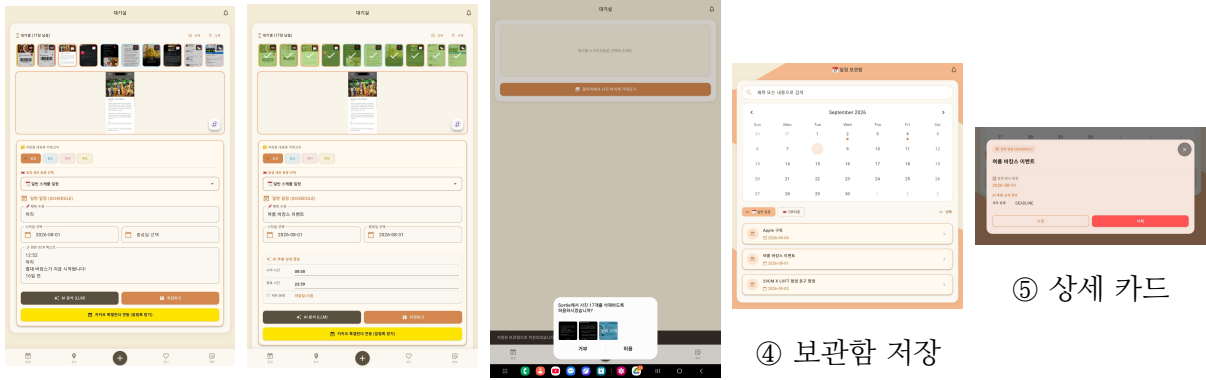
```
for (int i = 0; i < selected.length; i += 20) {
    final chunk = selected.sublist(
        i,
        (i + 20 > selected.length)
            ? selected.length
            : i + 20
    );
    ...
}
```

서버 측에서는 동시에 실행되는 LLM 호출 수를 3개로 제한한다. 따라서 “20장의 이미지를 한 번에 LLM에 동시에 무제한 전송”하는 방식이 아니라, 클라이언트의 배치 크기와 서버의 동시성 제한을 함께 사용하는 구조이다.

3.12. 사용자 검토 및 상태 관리

DB의 `status` 필드는 DRAFT 등의 상태를 표현하며, 모바일 DB에는 사용자가 확인한 결과를 CONFIRMED 상태로 저장하는 흐름이 구현되어 있다. 또한 `updateFields`, `updateTypeAndFields`, `updateStatus` 등의 CRUD 함수가 존재하여 사용자가 구조화 결과를 수정하거나 상태를 변경할 수 있다.

이 구조는 완전 자동화만을 목표로 하기보다 “자동 추출 → 사용자 확인/수정 → 저장”이라는 Human-in-the-loop 흐름을 지원한다.



① OCR 결과 ② AI 분석/수정 ③ 보존 여부

Figure 8: 일정(SCHEDULE) 데이터의 자동 추출, 사용자 검토 및 보관 시스템 흐름

4. 연구 결과 분석 및 평가

4.1. 평가 목적

평가는 두 가지 질문에 초점을 둔다.

첫째, 정형 패턴 기반 추출과 LLM 기반 추출은 어떤 종류의 필드에서 서로 다른 성능을 보이는가?

둘째, 두 방식의 처리 시간과 LLM 토큰 사용량에는 어느 정도 차이가 있는가?

본 실험은 프로젝트에 포함된 `evaluate_extraction.py`를 사용하였다. 카테고리 분류 자체는 실험 대상에서 제외하고 OCR 텍스트에서 세부 정보를 추출하는 능력에 집중하였다.

4.2. 평가 데이터 및 방법

평가 스크립트는 `dataset.csv`에서 OCR 텍스트 길이가 50자 이하인 항목을 제외한 뒤 네 카테고리에서 각각 최대 50개를 random seed 42로 샘플링한다. 따라서 본 실험은 SCHEDULE, PLACE, WISHLIST, MEMO 각각 50개, 총 200개를 대상으로 한다.

비교 대상은 다음과 같다.

- **Regex baseline:** 날짜, 가격, 장소 등에 대한 정규표현식 기반 추출
- **LLM:** GPT-4o-mini를 이용한 카테고리별 정보 추출
- **Reference annotation:** GPT-4o가 동일 OCR 텍스트에서 명시적 정보를 추출하여 생성한 기준값

여기서 GPT-4o 기준값은 사람이 독립적으로 작성한 정답지가 아니므로 본 보고서에서는 “ground truth” 대신 “reference annotation”이라고 표현한다.

4.3. 평가 기준

평가 스크립트는 문자열을 영문/숫자/한글 중심으로 정규화한 뒤 예측값과 기준값 사이에 포함 관계가 있는지를 검사한다. 따라서 정확한 문자열 동일성만 평가하는 것이 아니라 부분 매칭을 허용한다.

이 방식은 OCR 결과의 표현 차이를 어느 정도 흡수할 수 있지만, 동시에 “문자열 포함”만으로 정답을 판단하므로 의미적으로 완전히 동일한지를 평가하는 방법은 아니다. 따라서 결과는 “reference annotation 대비 매칭률”로 해석한다.

4.4. 카테고리별 처리 시간

카테고리	Regex 평균	LLM 평균	샘플 수
SCHEDULE	0.199 ms	1.317 s	50
PLACE	0.123 ms	0.889 s	50
WISHLIST	0.135 ms	0.784 s	50
MEMO	0.083 ms	1.230 s	50
전체 평균	0.135 ms	1.06 s	200

Table 6: 카테고리별 평균 처리 시간

처리 시간은 평가 스크립트에서 각 처리 단계의 시작과 종료 시점을 기록하여 측정하였다. Regex 처리 시간은 실제 정규표현식 기반 정보 추출 함수의 실행 시간이며, LLM 처리 시간은 LLM 호출에 소요된 시간을 측정한 값이다. 따라서 두 지표는 OCR, 이미지 로딩, 네트워크 전송, 데이터베이스 저장 및 화면 전환 등을 포함하는 전체 앱 처리 시간과는 구분된다.

측정 결과, Regex 기반 처리는 전체 평균 0.135ms로 매우 짧은 실행 시간을 보인 반면, LLM 기반 처리는 전체 평균 1.06s가 소요되었다. 이 값은 카테고리별 평균값을 기준으로 소수점 둘째 자리에서 반올림한 결과이다. 이는 사전에 정의된 패턴을 탐색하는 Regex 방식과 LLM 호출 및 응답 처리를 수행하는 방식의 처리 구조 차이에 따른 결과로 볼 수 있다.

카테고리별로는 Regex 처리 시간이 0.083–0.199ms 범위로 나타나 카테고리에 따른 차이가 크지 않았다. 반면 LLM 처리 시간은 WISHLIST에서 평균 0.784s로 가장 짧았고, SCHEDULE에서 평균 1.317s로 가장 길게 측정되었다. 이러한 차이는 카테고리별 입력 텍스트의 특성과 LLM 호출 및 응답 처리 과정의 차이에 영향을 받을 수 있다.

다만 본 측정값은 특정 처리 단계의 실행 시간을 나타내므로, 모바일 애플리케이션 전체의 종단간 처리 시간으로 해석하기보다는 각 처리 방식의 상대적인 실행 특성을 파악하기 위한 지표로 활용하는 것이 적절하다.

4.5. SCHEDULE 추출 성능

필드	Regex	LLM
날짜	66.7% (30/45)	33.3% (15/45)
가격	71.4% (10/14)	85.7% (12/14)
장소	9.5% (4/42)	66.7% (28/42)

Table 7: SCHEDULE의 reference annotation 대비 매칭률

※ 필드별 n 은 기준값(Reference Annotation)에 해당 필드가 존재하는 샘플의 수이다. ※ “-”는 Regex 방식에 해당 필드 추출 로직이 존재하지 않아 비교에서 제외됨을 의미한다.

SCHEDULE의 날짜에서는 Regex가 높은 매칭률을 보였다. 날짜는 2026.06.28, 2026년6월24일과 같이 비교적 규칙적인 문자열 패턴을 갖기 때문이다.

반면 장소에서는 LLM의 매칭률이 크게 높았다. 장소명은 “장소:”, “교환처:”와 같은 명시적인 키워드가 존재하지 않을 경우 단순 Regex만으로 추출하기 어렵기 때문이다. 가격 역시 LLM이 더 높은 매칭률을 보였으나, 가격은 Regex도 상당한 수준의 추출이 가능했다.

따라서 SCHEDULE 내부에서도 “날짜”와 “장소”는 동일한 알고리즘으로 처리할 필요가 없음을 확인할 수 있다.

4.6. PLACE 추출 성능

필드	Regex	LLM
상호명	5.6% (2/36)	80.6% (29/36)
지역/주소	–	89.7% (35/39)
가격	30.8% (4/13)	61.5% (8/13)

Table 8: PLACE의 reference annotation 대비 매칭률

※ 필드별 n 은 기준값(Reference Annotation)에 해당 필드가 존재하는 샘플의 수이다. ※ “–”는 Regex 방식에 해당 필드 추출 로직이 존재하지 않아 비교에서 제외됨을 의미한다.

PLACE에서는 Regex의 한계가 더욱 뚜렷하게 나타난다. 평가 스크립트의 Regex 장소 추출은 “장소”, “교환처”, “위치”, “가게명”, “호텔”, “숙소”, “매장” 등의 명시적 키워드 주변 문자열이나 at, @ 패턴을 대상으로 한다. 따라서 화면에 자연스럽게 나타난 “성수동 카페 추천”과 같은 텍스트에서 특정 상호명을 의미적으로 식별하는 것은 어렵다.

LLM은 이러한 문맥을 이용할 수 있으므로 상호명과 지역/주소에서 높은 매칭률을 보였다. 이는 LLM을 사용해야 하는 대표적인 정보 유형을 보여준다.

4.7. WISHLIST 추출 성능

필드	Regex	LLM
가격	87.8% (43/49)	95.9% (47/49)
상품명	–	64.3% (27/42)
브랜드	–	38.9% (7/18)

Table 9: WISHLIST의 reference annotation 대비 매칭률

※ 필드별 n 은 기준값(Reference Annotation)에 해당 필드가 존재하는 샘플의 수이다. ※ “–”는 Regex 방식에 해당 필드 추출 로직이 존재하지 않아 비교에서 제외됨을 의미한다.

WISHLIST의 가격은 Regex도 87.8%의 높은 매칭률을 보였다. 상품 가격은 통화 기호나 “원”과 같은 패턴이 비교적 명확하기 때문이다.

반면 상품명과 브랜드는 자연어 및 고유명사에 가까우므로 Regex baseline에서 별도의 추출 로직이 없어 비교가 어렵다. LLM은 상품명 64.3%, 브랜드 38.9%의 매칭률을 보였으며, 브랜드가 이미지 로고 형태로만 존재하거나 OCR에 포함되지 않는 경우에는 텍스트 기반 LLM만으로 복원하기 어려운 한계도 확인할 수 있다.

4.8. MEMO 추출 성능

필드	Regex	LLM
날짜	0.0% (0/2)	100.0% (2/2)
제목/핵심 정보	-	47.9% (23/48)

Table 10: MEMO의 reference annotation 대비 매칭률

※ 필드별 n 은 기준값(Reference Annotation)에 해당 필드가 존재하는 샘플의 수이다. ※ “-”는 Regex 방식에 해당 필드 추출 로직이 존재하지 않아 비교에서 제외됨을 의미한다.

기준값 내 날짜 정보가 포함된 표본이 2건으로 제한적이므로, 해당 100% 수치는 향후 대규모 데이터셋을 통한 추가 검증이 필요하다.

제목/핵심 정보는 비정형 텍스트의 요약 및 제목 생성 문제이므로 Regex baseline과 직접 비교하기 어렵다. LLM의 47.9% 매칭률은 해당 평가 로직에서 reference annotation과 부분적으로 일치한 비율이며, 향후 인간 평가와 추가 데이터셋을 통한 검증이 필요하다.

4.9. 종합 성능 비교

정보 유형	Regex의 특성	LLM의 특성
정형 날짜	빠르고 일정한 패턴에 강함	문맥 해석에 따른 누락 가능
정형 가격	높은 추출 가능	OCR 오류 및 문맥 보정 가능
장소/상호명	명시적 키워드가 있을 때 제한적	문맥 기반 식별에 강함
상품명	별도 의미 분석 없이는 제한적	자연어 문맥에서 추출 가능
주소/지역	다양한 표현 때문에 제한적	구조화 및 정규화와 결합 가능
제목/요약	일반적으로 부적합	의미 기반 처리 가능

Table 11: 정보 유형별 Regex와 LLM의 특성

본 결과에서 중요한 것은 “LLM이 항상 우수하다”는 결론이 아니다. 오히려 필드의 특성에 따라 적합한 처리 방식이 달라진다는 점이다. 날짜나 가격처럼 패턴이 명확한 데이터는 로컬 규칙 기반 처리의 장점이 크고, 장소명이나 상품명처럼 의미 해석이 필요한 데이터는 LLM의 장점이 나타난다.

따라서 솔티의 하이브리드 구조는 단일 모델의 우열을 전제로 하지 않고 서로 다른 처리기의 장점을 결합한다는 관점에서 설명하는 것이 적절하다.

일부 필드는 기준값에 해당 정보가 포함된 표본 수가 20건 미만으로 제한되어 있다. 이러한 소표본 필드들은 매칭률의 불확실성이 상대적으로 클 수 있으므로, 대규모 데이터셋을 이용한 추가 평가를 통해 결과의 안정성을 확인할 필요가 있다.

4.10. LLM 토큰 사용량 및 비용

평가 데이터에 기록된 LLM 평균 토큰 사용량은 카테고리별로 다음과 같다.

카테고리	평균 토큰/요청
SCHEDULE	452.12
PLACE	354.28
WISHLIST	332.18
MEMO	314.20
전체 평균	약 363

Table 12: 카테고리별 LLM 토큰 사용량

평가 결과 200건 처리에 총 72,639 tokens, 총 비용 약 \$0.044, 건당 약 0.3원이 소요된 것으로 나타났다. 본 수치는 2026년 8월 23일 평가 당시 측정된 GPT-4o-mini API 비용을 기준으로 산정된 것이며, 향후 서비스 운영 시점의 API 가격 정책에 따라 유동적일 수 있다.

4.11. 다중 처리 및 서버 동시성

클라이언트는 다중 이미지 요청을 최대 20개 단위로 분할한다. 서버의 Semaphore는 동시에 실행되는 LLM 호출을 3개로 제한한다.

다중 이미지 처리의 전체 wall-clock time과 실패율은 단건 처리와 별도의 배치 벤치마크를 통해 측정할 필요가 있다. 본 평가에서는 단건 Regex/LLM 처리 시간과 서버의 동시성 제어 구조를 중심으로 분석하였으며, 20건 일괄 처리의 종단간 성능은 향후 동일한 실행 환경에서 추가 평가할 대상으로 남겨두었다.

4.12. 개인정보 보호 평가

현재 시스템은 개인정보를 보호하기 위해 온디바이스 기반 초경량 NER 모델과 정규표현식(Regex)을 결합한 이중 방어막(Double Defense) 체계를 구축하였다.

초기 단순 정규표현식 기반 탐지는 극단적 비표준 텍스트에서의 문맥 유추 실패나 자연어 구문 기반의 심층적 의미 론 추론 부재, 잦은 유지보수 요구 등의 한계가 있었다. 이를 극복하기 위해 하이브리드 키워드 매칭 알고리즘을 고도화하여 오탐지(False Positive)를 차단하였으며, 나아가

텍스트 전체를 NER 언어 모델에 통과시켜 문맥 기반으로 개인정보를 1차로 가리고, 기존 암호화 정규식으로 2차로 덮는 이중 방어망을 전면 탑재하였다.

이러한 구조는 특정 키워드가 존재하지 않더라도 문장의 형태소 배열을 분석하여 개인정보일 확률을 산출할 수 있게 하며, 정규식의 보수적 탐지와 NER의 문맥 기반 탐지가 상호 보완되어 탐지 성능의 향상을 도모한다. 특히, 모든 추론 연산이 서버를 거치지 않고 사용자 기기 단말에서 독립적으로 수행되므로 민감 정보의 서버 전송을 최소화하고 구조적인 보안성을 높일 수 있다. 추후 다양한 변형 패턴을 포함한 데이터셋을 통해 정량적 탐지율을 검증할 계획이다.

4.13. 개발 과정의 문제와 개선

프로젝트 코드를 기준으로 확인되는 주요 개선 사례는 다음과 같다.

4.13.1. 기프트콘 OCR 텍스트 축약 문제

초기에는 기프트콘 텍스트에서 교환처, 상품명, 유효기간을 추출하여 간략한 세 줄의 텍스트로 만드는 방식이 존재했다. 그러나 이 방식은 원본 OCR의 다른 정보가 제거되어 LLM이 충분한 문맥을 볼 수 없고, 결과적으로 예제에 포함된 상호명을 잘못 사용하는 문제가 발생할 수 있었다.

현재 코드에서는 해당 축약 결과를 LLM 입력으로 사용하지 않고 원본 OCR 전체를 보존하도록 수정되어 있다. 이는 전처리가 오히려 후속 처리를 담당하는 AI에게 필요한 문맥 정보까지 과도하게 제거할 수 있는 문제를 개선한 사례이다.

4.13.2. 지역명 정규화 문제

장소 정보에서는 지역 필터링을 위해 표준 지역명을 사용한다. 특히 “해운대구”라는 문자열이 “대구”와 부분적으로 겹칠 수 있는 문제를 별도의 예외로 처리하고, 경상남도/경상북도/전라남도 등의 표현을 경남/경북/전라 등으로 정규화한다.

이러한 규칙은 LLM 출력 자체를 신뢰하는 것이 아니라, 애플리케이션이 실제로 필요로 하는 데이터 규격에 맞추는 후처리 계층의 역할을 보여준다.

4.13.3. LLM 출력의 불확실성

LLM 응답은 생성 결과이므로 필드 누락이나 허용되지 않은 타입이 반환될 가능성이 있다. validator는 이를 애플리케이션 수준에서 검사하고 누락 필드를 표시한다.

따라서 본 시스템은 “LLM의 응답을 곧바로 DB에 저장”하지 않고, 검증 및 enrichment를 거친 결과를 사용하는 구조로 개선되었다.

5. 결론 및 향후 연구 방향

5.1. 연구 결과 요약

본 연구에서는 스마트폰 스크린샷을 일정, 장소, 위시리스트, 메모 등 활용 가능한 구조화 데이터로 변환하는 솔티 애플리케이션을 구현하였다.

시스템은 Flutter 기반 모바일 클라이언트에서 Google ML Kit OCR을 수행하고, TFLite 기반 온디바이스 텍스트 분류기로 네 가지 대분류를 판단한다. 분류기의 입력은 프로젝트의 vocabulary와 IDF를 이용한 TF-IDF character n-gram 특징으로 구성된다.

개인정보 보호 측면에서는 NER와 Regex를 결합하여 전화번호, 주민등록번호, 카드번호, 계좌번호, 예약번호, 여권 및 MRZ 등 다양한 패턴을 탐지하고, 탐지된 문자열을 AES-256-CBC로 암호화하여 [ENC:...] 토큰으로 변환한다. 이를 통해 서버의 LLM이 원문의 민감 문자열을 직접 입력받지 않는 구조를 구현하였다.

서버에서는 FastAPI가 분석 요청을 처리하고, 카테고리별 LLM 프롬프트를 이용하여 일정, 장소, 상품, 메모 등의 필드를 구조화한다. Validator는 필수 필드와 타입을 확인하고 지역명, 가격, 알림 설정, 이미지 보존 정책 등 애플리케이션에 필요한 후처리를 수행한다.

저장 계층에서는 SQLite[?]와 JSON을 결합하여 카테고리별로 서로 다른 필드를 유연하게 저장한다. 또한 검색 및 필터링, 이미지 해시 기반 결과 조회, 알림 스케줄러, iCalendar 생성, Kakao Calendar 연동 등의 후속 기능을 구현하였다.

5.2. 정량 평가의 의의

200개 샘플을 이용한 정보 추출 평가에서 전체 평균 Regex 측정 시간은 0.135 ms, LLM 측정 시간은 약 1.06 s였다. Regex는 날짜와 가격처럼 정형적인 패턴에서 높은 매칭률을 보였으며, LLM은 장소명, 상호명, 주소 등 문맥 기반 정보에서 더 높은 매칭률을 보였다.

특히 SCHEDULE의 장소 매칭은 Regex 9.5% 대비 LLM 66.7%, PLACE의 상호명은 Regex 5.6% 대비 LLM 80.6%로 나타났다. 이러한 결과는 정형 정보와 비정형 정보에 동일한 추출기를 적용하는 것보다 정보 특성에 따라 역할을 분리하는 것이 합리적임을 보여준다.

본 평가의 기준값은 GPT-4o가 생성한 reference annotation이며, 결과는 해당 기준값 대비 매칭률로 해석한다.

5.3. 연구의 한계

첫째, OCR 자체의 정확도를 독립적으로 평가한 실험은 본 정보 추출 평가와 별개의 문제이다. OCR 오류가 발생하면 이후 분류와 구조화 단계에도 영향을 줄 수 있다.

둘째, 현재 평가셋은 카테고리별 50건, 총 200건으로 제한되어 있다. 다양한 앱 UI, 해상도, 언어 혼합, OCR 오류 유형을 충분히 대표한다고 보기 어렵다.

셋째, 개인정보 보호 로직은 코드상 다양한 패턴을 지원하지만 각 유형의 precision과 recall을 정량적으로 검증한 독립적인 보안 평가셋이 부족하다.

넷째, AES-256-CBC를 사용하고 있어 암호화된 데이터의 무결성 검증을 별도로 제공하지 않는다. 따라서 향후 authenticated encryption 방식으로 개선할 필요가 있다.

다섯째, 서버 기반 LLM을 사용하므로 네트워크 연결이 필요한 후처리 단계가 존재한다. 온디바이스 분류와 마스킹은 로컬에서 수행되지만 상세 구조화까지 완전한 오프라인으로 동작하는 것은 아니다.

여섯째, 현재 로컬 DB 저장 시 일부 구조화된 텍스트가 평문으로 저장되는 한계가 존재하므로, 향후 로컬 스토리지 전체에 대한 암호화(Data-at-Rest Encryption) 적용을 검토할 필요가 있다.

5.4. 연구 결과의 학술적·산업적 의의

본 연구를 통해 구현된 솔티(Sortie)는 비용 효율적인 온디바이스-클라우드 하이브리드 지능을 실증하였다는 점에서 학술적 의의가 있다. 기존 클라우드 의존형 AI 서비스가 안고 있는 천문학적 API 비용 및 프라이버시 침해 문제를 로컬 전처리와 이중 방어망(Double Defense PII) 아키텍처로 해결하였다. 동시에, 스마트폰 환경에서 사용자에게 가장 익숙한 캡처 행위를 '지능형 액션(Vision-to-Action)'으로 전환함으로써 높은 상용 경쟁력을 입증하였다.

5.5. 비즈니스 모델 및 확장 가능성

본 시스템은 단순한 생산성 도구를 넘어 지속 가능한 비즈니스 모델(BM)로 확장할 수 있는 강력한 잠재력을 지닌다.

5.5.1. B2C 제휴 커머스(Affiliate) 및 타겟 마케팅

사용자가 캡처한 위시리스트나 장소 데이터는 구매 및 방문 의도가 가장 명확히 담긴 '초기 관심사(Pre-search) 데이터'이다. 현재 구현된 네이버 쇼핑 및 지도 연동을 쿠팡 파트너스, 캐치테이블, 야놀자 등의 제휴 마케팅 API와 직접 결합할 경우, 사용자가 버튼을 클릭하여 발생하는 전환(Conversion)에 대해 수수료(Commission)를 취득하는 즉각적인 수익 창출이 가능하다.

5.5.2. B2B 엔터프라이즈 SDK 라이선싱

솔티의 핵심 모듈인 '초경량 온디바이스 분류 및 개인정보 암호화 엔진'을 모듈화하여 금융 앱이나 핀테크 서비스에 B2B SDK 형태로 라이선싱할 수 있다. 예를 들어, 사용자가 캡처한 고지서나 송금

화면에서 민감정보 노출 없이 필요한 결제 데이터만 안전하게 구조화하여 핀테크 앱으로 넘겨주는 식의 엔터프라이즈 결합이 가능하다.

5.5.3. OS 통합형 지능형 비서 (Agentic Gateway)

향후 모바일 OS(Android, iOS)의 '공유하기(Share Sheet)' 시스템 수준으로 솔티를 통합할 수 있다. 사용자가 스크린샷을 찍는 즉시 백그라운드에서 솔티가 데이터를 구조화하고, 알림창으로 "지도에서 볼까요?", "캘린더에 등록할까요?"를 선제적으로 제안하는 '선행 액션 관문(Agentic Gateway)'으로의 생태계 확장이 목표이다.

5.5.4. 보안 및 AI 기술의 고도화

기술적으로는 현재의 AES-CBC 암호화 방식을 무결성 검증이 가능한 AES-GCM 방식으로 고도화하고, 장기적으로 스마트폰 자체에 경량화된 sLLM(Small Large Language Model)을 오프라인으로 탑재하여 네트워크 단절 환경에서도 심층 구조화를 지원하는 완전한 프라이버시 보장형(Privacy-Preserving) 온디바이스 AI로 발전시켜 나갈 계획이다.

6. 참고 문헌

- [1] M. Goyal, D. Garg, R. S. Munjal, D. P. Mohanty, S. Moharana, and S. P. Thota, "ScreenSeg: On-Device Screenshot Layout Analysis," arXiv preprint arXiv:2104.08052, 2021.
- [2] S. Garg, Harichandana, and S. Kumar, "On-Device Document Classification using multimodal features," *Proceedings of the 8th ACM IKDD CODS and 26th COMAD (CODS COMAD 2021)*, pp. 1-5, 2021.
- [3] G. Shim, S. Hong, and H. Lim, "REVISE: A Framework for Revising OCRred text in Practical Information Systems with Data Contamination Strategy," arXiv preprint arXiv:2604.08115, 2026.
- [4] 이종한, 박성재, 이채은, 권아현, 김민재, 신현석, 황연우, 서왕덕, "개인정보 보호형 LLM 기반 실시간 근로계약서 분석 및 가이드 시스템," *2026 한국정보기술학회 하계 종합학술대회 논문집*, 2026.
- [5] 정휘수, 이지훈, 주성진, 이길호, "On-device LLM모델의 반응성 향상을 위한 동적 배치 최적화 기술," *2024 한국소프트웨어종합학술대회 논문집*, 2024.
- [6] B. Chen, Y. Chen, X. Jiang, et al., "Unleashing the Potential of Prompt Engineering in Large Language Models: A Comprehensive Review," arXiv preprint arXiv:2310.14735, 2023.
- [7] Google, "ML Kit Text Recognition Documentation," Google Developers. <https://developers.google.com/ml-kit/vision/text-recognition> (2026년 9월 8일 접속)
- [8] Flutter, "Flutter Documentation," <https://docs.flutter.dev/> (2026년 9월 8일 접속)
- [9] Pydantic, "Pydantic Documentation," <https://docs.pydantic.dev/> (2026년 9월 8일 접속)
- [10] OpenAI, "OpenAI API Documentation," <https://platform.openai.com/docs/> (2026년 9월 8일 접속)
- [11] SQLite, "SQLite Documentation," <https://www.sqlite.org/docs.html> (2026년 9월 8일 접속)